

LEM1_9 Variants
James Brakefield
Brakefield Research © 2014

2) LEM: Logic Emulation Machine

2.a) Characterization:

1-bit data, 9-bit instructions
Bit accumulator and carry bit
Data memory consists of single bits
Instructions from block RAM: 1024 by 9 for Altera, also 2048 by 9 for Xilinx
Absolute addressing only
Goals: small LUT count (<100), high frequency of operation, one or two pipeline stages

2.b) Instance: LEM1_9min:

Supports 2K instruction ROM and 64 bit data RAM
Bit instructions to/from accumulator
 XXXnnnnnnn
Also bit operations between accumulator and carry bit
 0000xxxxx
No branches or conditional control flow
Implemented in 2003
Device independence in 2014

2.c) Instance: LEM1_9accum:

Supports 2K instruction ROM and 64 bit data RAM
Bit instructions to/from accumulator
 XXXnnnnnnn
Also bit operations between accumulator and carry bit
 00000xxxxx
Conditional branches and call using two word instructions
 000xxxxnnn nnnnnnnnnn
Call and return via very small address stack

2.d) Instance: LEM1_9stk:

Supports 2K instruction ROM and 64 bit data RAM
Bit instructions to/from top of bit stack
 XXXnnnnnnn
Also bit stack operations
 0000xxxxx
Conditional branches and call via two word instructions
 000xxxxnnn nnnnnnnnnn
Call and return via very small address stack

2.e) Instance: LEM1_9stk:

Supports 2K instruction ROM and 32 to 256 bits of data RAM
Bit instructions to/from top of bit stack of 16+ locations
Return address stack of 16+ locations
 XXXRnnnnnn: R is 1 for a replace operation (pop on replace op)
 000Rnnnnnn: LD/STP (load & push, store & pop)
 001Rnnnnnn: LDC/ST (load complement & push, store & no pop)
 010Rnnnnnn: AND (and to TOS)
 011Rnnnnnn: OR (or to TOS)
 100Rnnnnnn: XOR (xor to TOS)
 101Rnnnnnn: ADC (add with carry to TOS)
 1100nnnnnn: increment memory by carry
 1101nnnnnn: decrement memory by carry
 1110xxxxaa aaaaaaaaaa: CALL/JMP/JCC/JCS/JA0/JA1

111011nnn nnnniiiiR: data memory and op-code expansion
 1111sccaa: swap X nop/complement/clear/set X C/A
 111100000: for no-swap, C/A both NOP is the return instruction
 XXXR00nnn is top of stack relative
 XXXR01nnn is offset for data pointer?
 XXXR1nnnn is data RAM

Memory allocation for scrolling 7-segment "Hello World"

000xxx stack locations 1..5
 000101 bit at address pointer location
 000110 bit at address pointer location with adr auto inc
 000111 bit at address pointer location with adr auto dec
 001xxx 8-bit address pointer

0100xx 4-bit 7-segment decode register (eight outputs)
 0101xx 1st digit on display (low end of large counter)
 01100x digit select (higher up on large counter)
 011010 enable for digit select

Other working data RAM

2.f) Instance: LEM1_9ptr:

Supports 64+ word instruction ROM and 32 to 256 bits of data RAM

Bit instructions to/from top of bit stack of 4+ locations

Three bit RAM pointers with auto inc/dec using 9 data memory locations

Return address stack of 4+ locations

Parameterization: return address stack depth, instruction address width, bit RAM size, bit stack size

XXXRnnnnnn: R is 1 for a replace operation (pop on replace op)
 000Rnnnnnn: LD/STP (load & push, store & pop)
 001Rnnnnnn: LDC/ST (load complement & push, store & no pop)
 010Rnnnnnn: AND (and to TOS)
 011Rnnnnnn: OR (or to TOS)
 100Rnnnnnn: XOR (xor to TOS)
 101Rnnnnnn: ADC (add with carry to TOS)
 1100nnnnnn: increment memory by carry
 1101nnnnnn: decrement memory by carry
 1110xxxxaa aaaaaaaa: CALL/JMP/JCC/JCS/JA0/JA1
 111011nn nnnniiiiR: data memory and op-code expansion
 111011000 0nnnnnnnn: initialize stack pointer
 1110110xx 0nnnnnnnn: initialize pointer xx
 1110110xx 1nnnnnnnn: unused
 1111sccaa: {null/swap}x{nop/complement/clear/set}x{C/A}
 111100000: for no-swap and C/A both NOP is the return instruction
 XXXR00nnn is top of stack relative
 XXXR01nnn is offset for data pointer?
 XXXR1nnnn is data RAM

Pointer mechanism

000xxx stack locations 1..5
 000101 bit at address 1st pointer location
 000110 bit at address 1st pointer location with adr auto inc
 000111 bit at address 1st pointer location with adr auto dec
 001000
 001001 bit at address 2nd pointer location
 001010 bit at address 2nd pointer location with adr auto inc
 001011 bit at address 2nd pointer location with adr auto dec
 001100
 001101 bit at address 3rd pointer location
 001110 bit at address 3rd pointer location with adr auto inc

001111	bit at address 3 rd pointer location with adr auto dec
Memory allocation	for scrolling 7-segment "Hello World"
0100xx	4-bit 7-segment decode register (eight outputs)
0101xx	1 st digit on display (low end of large counter)
01100x	digit select (higher up on large counter)
011010	enable for digit select
Other	working data RAM

2.g) Instance: LEM1_9stg:

Bit string dominated ISA:

xxxxrpssuu:	instructions using pointers (ss != 11)
xxxxrp1100 nnnnnnnnn:	instructions with absolute bit addresses
xxxxrp11uu nnnnnnnnn:	instructions with pointer offset
xxx	op-code (move, and, or, xor, adc, andc, movc)
111	other op-codes (call, branch, load pointer, expansion)
r	replace, e.g., result to memory
p	enable pop/push ("r" selects pop/push) stack
ss	pointer mode: (uu), (uu)++, (--uu), (uu)+nnnnnnnnn
uu	pointer selection (stack is pointer #0)
1110xaacc:	set/clear/swap A and/or C, return,
1111sccaa:	{null/swap}x{nop/complement/clear/set}x{C/A}
111100000:	for no-swap and C/A both NOP is the return instruction
111111c1a:	four available/redundant op-codes
Load/adjust pointers, call & branch, instructions using memory adrs	
1111xxxxaa aaaaaaaaa:	CALL/JMP/JCC/JCS/JA0/JA1
1111110uu nnnnnnnnn:	initialize data/stack pointer
1111111uu nnnnnnnnn:	adjust data/stack pointer

2.h) Instance: LEM4_9stk:

Usable as a four-function calculator

Supports 2K instruction ROM and 32 to 256 digits of data RAM

4-bit data operations to/from top of digit stack of 16+ locations

4LUT instruction, small return address stack

Return address stack of 16+ locations

XXXXRnnnnn:	R is 1 for a replace operation (pop on replace op)
000Rnnnnn:	LD/STP (load & push, store & pop)
001Rnnnnn:	LDC/ST (load complement & push, store & no pop)
010Rnnnnn:	AND (and to TOS)
011Rnnnnn:	OR (or to TOS)
100Rnnnnn:	XOR (xor to TOS)
101Rnnnnn:	ADC (add with carry to TOS)
1100nnnnn:	increment memory by carry
1101nnnnn:	decrement memory by carry
1110xxxxaa aaaaaaaaa:	CALL/JMP/JCC/JCS/JAZ/JANZ
111011xaa aaaaaaaaa:	JPL/JMI or by left shift/rotate thru carry?
1110110aa aaaaaa0000:	4-bit LUT instruction?
1110110nn nnnnniiii:	rotate/shift instructions (i != 0)?
1110111nn nnnnniiiR:	data memory expansion?
11110xxxx:	LDI
11111ccaa:	nop/complement/clear/set C/A
111110000:	both C & A nop is the return instruction
XXXXR00nnn	is top of stack relative
XXXXR01nnn	is offset for data pointer?
XXXXR1nnnn	is data RAM

Memory allocation for scrolling 7-segment "Hello World"

000xxx	stack relative locations
--------	--------------------------

00100x	4-bit 7-segment decode registers (eight outputs)
00101x	8-bit address pointer (unused)
001100	digit at address pointer location (unused)
001101	BCD enable, 7-seg enable, digit sel (from counter)
001110	digit at address pointer location with adr auto inc
001111	digit at address pointer location with adr auto dec
010000	1 st digit on display (low end of large counter)
Other	working data RAM

2.h) Instance: LEM1_9risc:

Supports 2K instruction ROM and 512 bit data RAM

Bit instructions among eight bit “registers”

XXXdddsss

Load/store instructions

11Xnnnnnn

Conditional branches and call using two word instructions

000xxxxnn nnnnnnnnn

Call and return via very small address stack

3.b) Instance: LEM1_9min:

Supports 2K instruction ROM and 64 bit data RAM

Bit instructions to/from accumulator

XXXnnnnnn

Also bit operations between accumulator and carry bit

0000xxxxx

No branches or conditional control flow

opHLT	"000000000"	wait for system clock
opCACC	"000010000"	clear A, clear C
opCASC	"000010001"	clear A. set C
opSACC	"000010010"	set A, clear C
opSASC	"000010011"	set A, set C
opCC	"000010100"	clear C
opSC	"000010101"	set C
opCA	"000010110"	clear A
opSA	"000010111"	set A
opOR2C	"000011000"	A C to C
opNA	"000011001"	negate A
opNC	"000011010"	negate C
opNANC	"000011011"	negate A, negate C
opAND2C	"000011100"	A & C to C
opC2A	"000011101"	copy C to A
opA2C	"000011110"	copy A to C
opXAC	"000011111"	swap A and C
opST	"001nnnnnn"	store A at memory location "nnnnnn"
opLD	"010nnnnnn"	load A from memory location
opLDC	"011nnnnnn"	load A complement from memory location
opAND	"100nnnnnn"	AND memory location into A
opOR	"101nnnnnn"	OR memory location into A
opXOR	"110nnnnnn"	XOR memory location into A
opADC	"111nnnnnn"	sum to A, carry to C

3.c) Instance: LEM1_9accum:

Supports 2K instruction ROM and 64 bit data RAM

Bit instructions to/from accumulator

XXXnnnnnnn

Also bit operations between accumulator and carry bit

00000xxxx

Conditional branches and call using two word instructions

000xxxxnnn nnnnnnnnn

Call and return via very small address stack

opNOP	"000000000"	waste a clock cycle
opRTN	"000000001"	pop return stack into PC
opCACC	"000010000"	clear A, clear C
opCASC	"000010001"	clear A, set C
opSACC	"000010010"	set A, clear C
opSASC	"000010011"	set A, set C
opCC	"000010100"	clear C
opSC	"000010101"	set C
opCA	"000010110"	clear A
opSA	"000010111"	set A
opOR2C	"000011000"	A C to C
opNA	"000011001"	negate A
opNC	"000011010"	negate C
opNANC	"000011011"	negate A, negate C
opAND2C	"000011100"	A & C to C
opC2A	"000011101"	copy C to A
opA2C	"000011110"	copy A to C
opXAC	"000011111"	swap A and C
opBR	"0001000nn" "nnnnnnnnnn"	branch to location "nnnnnnnnnn"
opCALL	"0001001nn" "nnnnnnnnnn"	branch to location "n", save PC
opBRA0	"0001010nn" "nnnnnnnnnn"	branch to location "n" if A zero
opBRA1	"0001011nn" "nnnnnnnnnn"	branch to location "n" if A one
opBRC0	"0001110nn" "nnnnnnnnnn"	branch to location "n" if C zero
opBRC1	"0001111nn" "nnnnnnnnnn"	branch to location "n" if C one
opST	"001nnnnnn"	store A at memory location "nnnnnn"
opLD	"010nnnnnn"	load A from memory location
opANDC	"011nnnnnn"	AND complement of memory location into A
opAND	"100nnnnnn"	AND memory location into A
opOR	"101nnnnnn"	OR memory location into A
opXOR	"110nnnnnn"	XOR memory location into A
opADC	"111nnnnnn"	sum to A, carry to C

3.d) Instance: LEM1_9stk:

Supports 2K instruction ROM and 64 bit data RAM

Bit instructions to/from top of bit stack

XXXnnnnnnn

Also bit stack operations

0000xxxxx

Conditional branches and call via two word instructions

000xxxxnn nnnnnnnnn

Call and return via very small address stack

A is top of data stack and C is second from top on data stack

opNOP	"000000000"	waste a clock cycle
opRTN	"000000001"	pop return stack into PC
op0	"000010000"	push 0 to A
op1	"000010001"	push 1 to A
opnA	"000010010"	complement A
opDUP	"000010011"	duplicate and push A
opPOP	"000010100"	pop A
opSWAP	"000010101"	swap A and C
op	"000010110"	
op	"000010111"	
opXOR	"000011000"	pop C, A xor C to A
opXORC	"000011001"	pop C, not A xor C to A
op	"000011010"	
op	"000011011"	
opOR	"000011100"	pop C, A C to A
opORC	"000011101"	pop C, not A C to A
opAND	"000011110"	pop C, A & C to A
opANDC	"000011111"	pop C, not A & C to A
opBR	"0001000nn" "nnnnnnnnnn"	branch to location "nnnnnnnnnn"
opCALL	"0001001nn" "nnnnnnnnnn"	branch to location "n", save PC
opBR0P	"0001010nn" "nnnnnnnnnn"	branch to location "n" if A zero, pop A
opBR1P	"0001011nn" "nnnnnnnnnn"	branch to location "n" if A one, pop A
opBR0	"0001110nn" "nnnnnnnnnn"	branch to location "n" if A zero
opBR1	"0001111nn" "nnnnnnnnnn"	branch to location "n" if A one
opST	"001nnnnnn"	store A at memory location "nnnnnn", pop A
opLD	"010nnnnnn"	push A, load A from memory location
opANDC	"011nnnnnn"	AND complement of memory location into A
opAND	"100nnnnnn"	AND memory location into A
opOR	"101nnnnnn"	OR memory location into A
opXOR	"110nnnnnn"	XOR memory location into A
opADC	"111nnnnnn"	sum to A, carry to C

3.e) Instance: LEM1_9risc:

Supports 2K instruction ROM and 512 bit data RAM

Bit instructions among eight bit "registers"

XXXdddsss

Load/store instructions

010ddd0xx nnnnnnnnn

Conditional branches and call using two word instructions

001ddd1nn nnnnnnnnn

001xxx1nn nnnnnnnnn

Call and return via very small address stack

opNOP	"000000000"	waste a clock cycle
opRTN	"000001000"	pop return stack into PC
	6 unused codes	
op02D	"000ddd001"	clear D
op12D	"000ddd010"	set D
opDC	"000ddd011"	negate D
	4*8 unused codes	
opBRA0	"001ddd0nn" "nnnnnnnnnn"	branch to location "n" if D zero
opBRA1	"001ddd1nn" "nnnnnnnnnn"	branch to location "n" if D one
opBR	"0100001nn" "nnnnnnnnnn"	branch to location "nnnnnnnnnnnn"
opCALL	"0100011nn" "nnnnnnnnnn"	branch to location "n", save PC
	6*4 unused codes	
opLD	"010ddd000" "nnnnnnnnnn"	load D from memory location "n"
opLDC	"010ddd001" "nnnnnnnnnn"	memory location "n" negated to D
opST	"010ddd010" "nnnnnnnnnn"	store D to memory location "n"
opSTC	"010ddd011" "nnnnnnnnnn"	store not D to memory location "n"
opANDC	"011dddsss"	AND complement of S into D
opAND	"100dddsss"	AND S into D
opOR	"101dddsss"	OR S into D
opXOR	"110dddsss"	XOR S into D
opADC	"111dddccc" "nnnnnnnnnn"	sum to D, carry to C